

1 Planning 策略——ReAct + Plan&Execute 混合模式

LeisureAgent 由 18 个 LangGraph 节点组成，按 ReAct（推理-行动-观察）和 Plan&Execute（计划-执行）混合运行。LLM 调用失败时自动降级规则逻辑。

1.1 工作流与路由

```
load_session → classify_intent [路由]
casual/out_of_domain → direct_reply → END
inquiry → search_inquiry → present_inquiry → END
new_plan → analyze_goal → search_candidates → detect_exceptions
(critical_gap) adjust_search → search_candidates (loop, 2)
→ compose_plan → persist_plan
(auto) execute_bookings → finalize_executed → END
(manual) present_plan → END
feedback → analyze_feedback → (search?) → compose → ... → END
confirm → execute_bookings
(failure) replan_execute → persist → execute (loop, 2)
(ok) finalize_executed → END
```

意图分类采用三层优先级：快速规则（确认/反馈关键词 + pending 方案 → 直接路由，工作日关键词 → rejection）；LLM 分类（轻量模型，temperature=0, max_tokens=512）；TF-IDF 语义匹配降级（char n-gram + 余弦相似度，支持模糊匹配如”哈喽”→CASUAL、”推荐好吃的”→INQUIRY）。

场景驱动编排：三种场景（family/friends/other）各有独立候选优先级.family 优先 amusement_park > scenic_spot，偏好火锅/中餐；friends 优先 exhibition_hall > scenic_spot。LLM 生成方案后经四层保障：

- (1) **JSON 严格校验**：LLM 输出 → 提取 → 清洗（单引号/尾部逗号/中文标点）→ Pydantic 校验，最多 3 次重试；
- (2) **多日校验**：day_num 分布不满足 day_count → LLM 重试 → 规则强制拆分；
- (3) **结构安全网**：LLM 产出缺失 dining/play → 从候选池按场景优先级自动补齐；
- (4) **LLM 降级**：调用失败 → 场景配置驱动的规则编排（_SCENARIO_CONFIG）。

编排硬约束：活动时序固定（游玩 → 缓冲 → 用餐）；用餐不重复（用户指定优先）；停留 5-360 min；多日每天 1 项。

2 工具调用链路

链路	输入	关键步骤	输出
搜索 (search)	constraints	遍历 restaurant/scenic_spot/amusement_park/exhibition_hall/mall 五类表筛选： _extract_named_locations() 强制纳入用户提及地点	candidates: dict[category, list[item]]
异常检测 (detect)	candidates	_check_availability() 检查容量： 判断 critical_gaps → 触发搜索自愈（≤2 次）	exceptions + warnings
持久化 (persist)	AgentPlan	创建 travel_plan 主记录： for each item: 场馆存在 → 名额 → 时段冲突检测（仅同 day_num 内）→ INSERT； 失败项保留在内存方案，WARNING 日志	persisted plan (DB + 内存)
执行预约 (execute)	plan_id	无需预约 → skip；无容量 → 直接标记； 有容量 → BEGIN IMMEDIATE + 原子 UPDATE WHERE current < max (row-count=0 判满)； 失败 → 同类替代重试（≤2 次，不调 LLM）	booking_results

表 1：核心工具调用链路

咨询模式：用户输入含领域关键词但非规划意图 → search_inquiry → present_inquiry（返回 InquiryEvent，前端展示 Yes/No/Other 弹窗）。

并发安全：SQLite 单连接 + threading.Lock() 写锁 + WAL 模式（并发读、串行写）。预约使用 BEGIN IMMEDIATE + 原子 SQL，杜绝 TOCTOU 竞态。

3 异常处理机制

架构总览：React (HashRouter + SSE) → FastAPI (/api/*) → LangGraph (18 nodes) → Tools (search/booking/location) → Service/Repository → SQLite (WAL+ 写锁)

机制	覆盖节点	处理策略
LLM 降级	classify / analyze / compose	LLM 失败 → 规则兜底：TF-IDF 语义匹配 (classify)、默认约束 (analyze)、场景配置驱动 (compose)。关键路径不依赖 LLM 可用性。
ReAct 搜索自愈	detect → adjust → search	关键类别无可用候选 → 放宽距离约束 50% → 重新搜索。上限 2 次，耗尽后 gap_report 告知用户缺口。
ReAct 执行自愈	execute → replan → persist → execute	预约失败 → 从原始 candidates 找同类替代 (不调 LLM) → 重新持久化执行。上限 2 次。
结构安全网	compose	LLM 产出缺少 dining/play → 从候选池按场景优先级确定性补齐，每次触发记录 safety_net 指标。
输入安全	load_session	22 个正则 (中英文) 检测：指令覆盖/角色劫持/提示词提取/SQL 注入标记。控制字符 + 零宽字符清洗。2000 字符上限。拦截后返回 guard_reject 事件。
结构化输出	invoke_structured	JSON 提取 → 语法清洗 → Pydantic 校验 → 业务校验 → 错误回传 LLM 重试。最多 3 次。
配置校验	启动时	validate_config() 检查 API key 等必需配置，缺失提前 WARNING。
可观测性	全局	JSON 结构化日志 + LLM 调用追踪 (节点/耗时/token) + 安全网计数。持久化到 SQLite metrics_counter 表，GET /api/agent/metrics 端点。

表 2：八层异常处理机制